Programación declarativa

• Haskell •

Doble grado en Ingeniería Informática y Matemáticas
Universidad Complutense de Madrid
Óscar Martín — Curso 2026/27

22. Evaluación perezosa

Pereza e impaciencia

- Dada una expresión, la evaluación perezosa, la impaciente y cualquier otro orden de evaluación siempre dan el mismo resultado… si terminan.
  – *teorema de Church-Rosser*
- Si hay un orden de evaluación que termina, la evaluación perezosa termina.

```
inf :: Int
inf = 1 + inf
```

- Con evaluación perezosa: `fst (0, inf)` $\rightsquigarrow$ 0.
- Con evaluación impaciente no termina.
- En `ghci`, `:sprint` muestra lo que ya se ha evaluado de un valor.

```
ghci> x = 3 + 4 :: Int
ghci> :sprint x
x = _
ghci> x
7
ghci> :sprint x
x = 7
```

- `:sprint` solo funciona con valores monomórficos.

```
ghci> x = 3 + 4
ghci> :sprint x
x = _
ghci> x
7
ghci> :sprint x
x = _
```

- Otros ejemplos:

```
ghci> xs = [2+3, 1, abs 4, 5-5] :: [Int]
ghci> :sprint xs
xs = [_,1,_,_]
ghci> xs !! 2
4
ghci> :sprint xs
xs = [_,1,4,_]
ghci> length xs
4
ghci> :sprint xs
xs = [_,1,4,_]
```

- Estas cosas que se muestran aquí como `_` se llaman técnicamente *thunks*.
- Algunas funciones de Haskell son impacientes en uno o varios de sus argumentos:
  - `&&`, `||`, `+`, `*`, `foldl'`, `const`…

```
ghci> b = 1 * 0 :: Int
ghci> d = 0 * b
ghci> :sprint b
b = _
ghci> d
0
ghci> :sprint b
b = 0
```

```
ghci> b = True && False
ghci> d = False && b
ghci> :sprint b
b = _
ghci> d
False
ghci> :sprint b
b = _
```

```
ghci> m = 1 + 1 :: Int
ghci> n = 3 + 3 :: Int
ghci> :sprint m
m = _
ghci> :sprint n
n = _
ghci> const m n
2
ghci> :sprint m
m = 2
ghci> :sprint n
n = _
```

- El ajuste de patrones puede requerir cierto nivel de evaluación.

```
siempre, cero :: Int -> Bool
siempre _ = True
cero 0 = True
cero _ = True
```

```
ghci> x = 3 + 4 :: Int
ghci> siempre x
True
ghci> :sprint x
x = _
ghci> cero x
True
ghci> :sprint x
x = 7
```

Compartición de expresiones

- La evaluación perezosa naíf puede hacer que ciertas subexpresiones se evalúen más de una vez.
  - Ejemplo: `cuadr (1+2)` ↝ `(1+2) * (1+2)`.
- Haskell hace compartición de expresiones.

```
ghci> x = 1 + 2 :: Int
ghci> p = (x, x)
ghci> :sprint p
p = (_,_)
ghci> fst p
3
ghci> :sprint p
p = (3,3)
```

```
ghci> y = 3 + 4 :: Int
ghci> q = (y, 3 + 4 :: Int)
ghci> fst q
7
ghci> :sprint q
q = (7,_)
```

- Cuando se habla de *evaluación perezosa* se entiende que incluye compartición.

`undefined` (fuera de menú)

- *Todos* los tipos en Haskell contienen el valor $\bot$.
- Hay varias formas equivalentes de escribirlo:
  - `undefined :: a`
  - `error :: String -> a`
    - `error "mensaje adecuado"`
  - evaluaciones que no terminan
    - `inf` (el de arriba)
  - `f x`, si la función `f` no está definida para el valor `x`
  - Todos ellos son semánticamente equivalentes: $\bot$.
- No es un problema hasta que se intenta evaluar:
  - `undefined` termina con un mensaje de error genérico;
  - `error` termina con el mensaje que recibe como parámetro;
  - `inf` no termina;
  - `f x` termina con un error de patrones no exhaustivos.

```
par :: (Int, Int) -> Bool
par (_, _) = True
```

```
ghci> par (undefined, error "¡Ay!")
True
ghci> par undefined
*** Exception: Prelude.undefined
...
```

Ejercicios

1. Las funciones `f1` y `f2` de abajo coinciden sobre valores definidos. ¿Cuáles de estas expresiones terminan?:

- `f1 False (inf > 0)`
- `f1 (inf > 0) False`
- `f2 False (inf > 0)`
- `f2 (inf > 0) False`

```
f1, f2 :: Bool -> Bool -> Bool
f1 False _ = False
f1 True  x = x
f2 _ False = False
f2 x True  = x
```

2. Predice qué mostrarán estos `:sprint` antes de ejecutarlos.

```
ghci> ys = map (*2) [1..5] :: [Int]
ghci> ys !! 0
ghci> :sprint ys
ghci> length ys
ghci> :sprint ys
```

3. En `ghci`, ejecutando `:set +s`, compara el tiempo y la necesidad de memoria de

- `let s = sum [1..10^7 :: Int] in s + s`

con los de

- `sum [1..10^7 :: Int] + sum [1..10^7 :: Int]`.